CS 31204: Computer Networks – The Data Link Layer

Department of Computer Science and Engineering

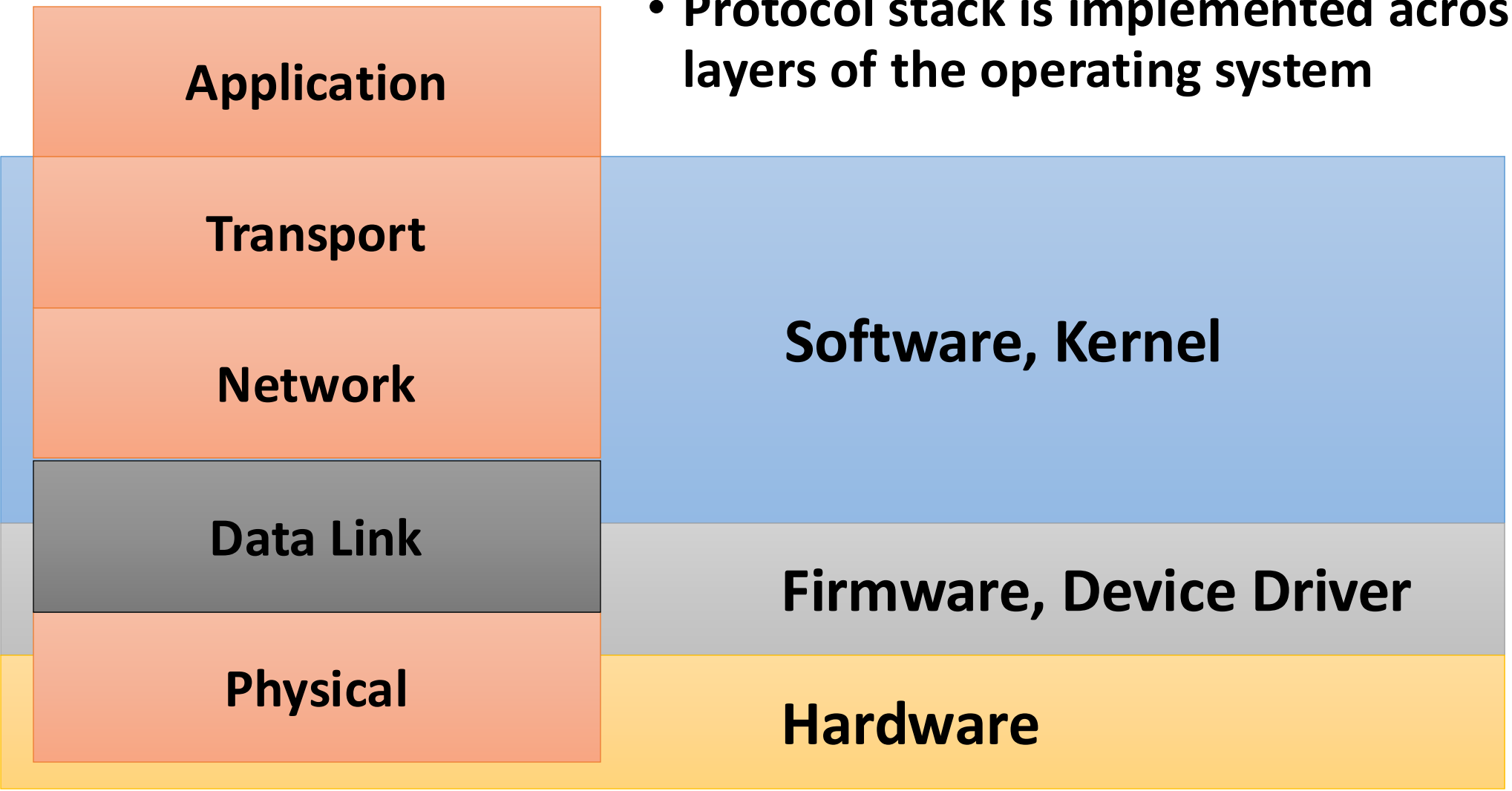
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

Abhijnan Chakraborty
abhijnan@cse.iitkgp.ac.in

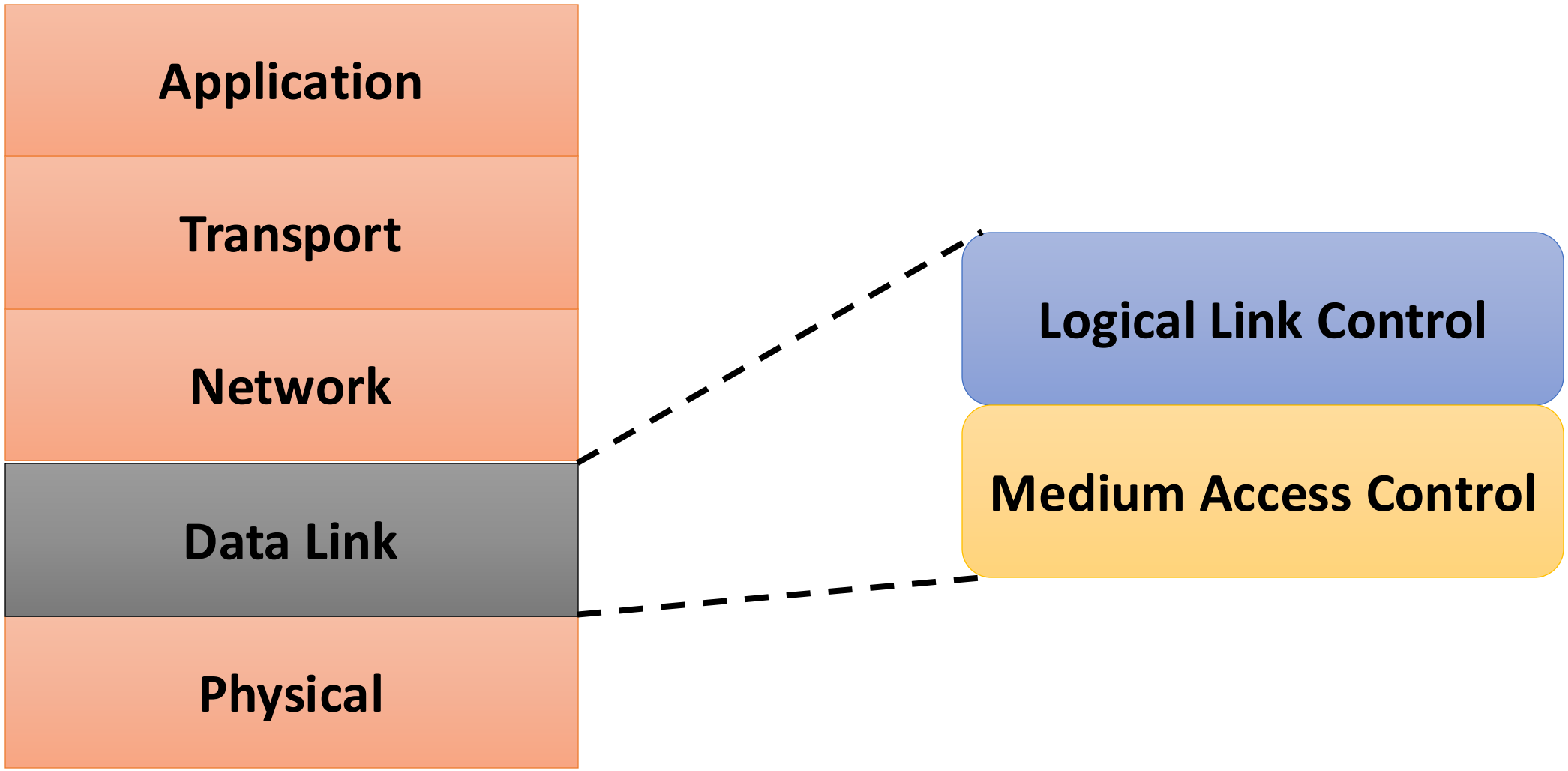
Sandip Chakraborty
sandipc@cse.iitkgp.ac.in

What We Have Learnt So Far …

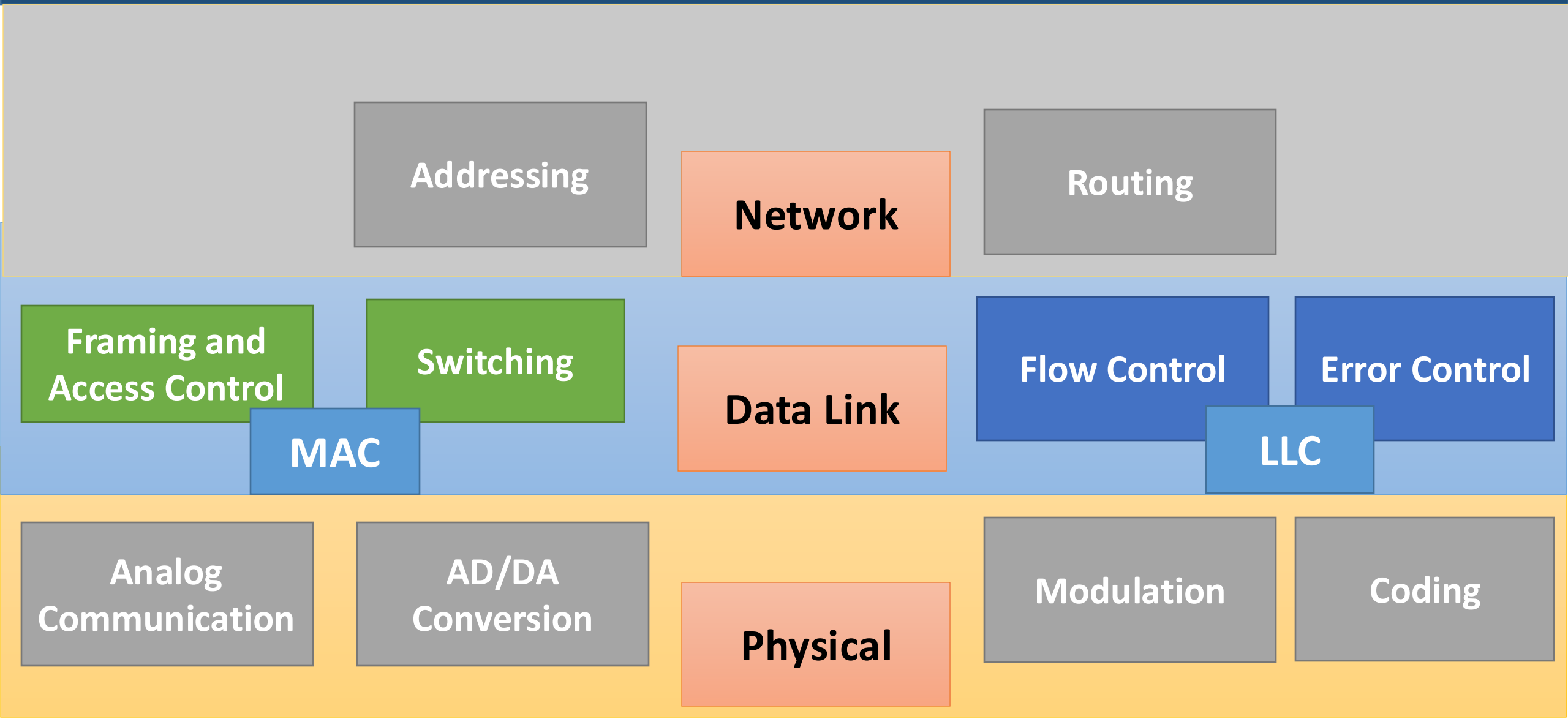
- Protocol stack is implemented across different layers of the operating system



The Data Link Layer

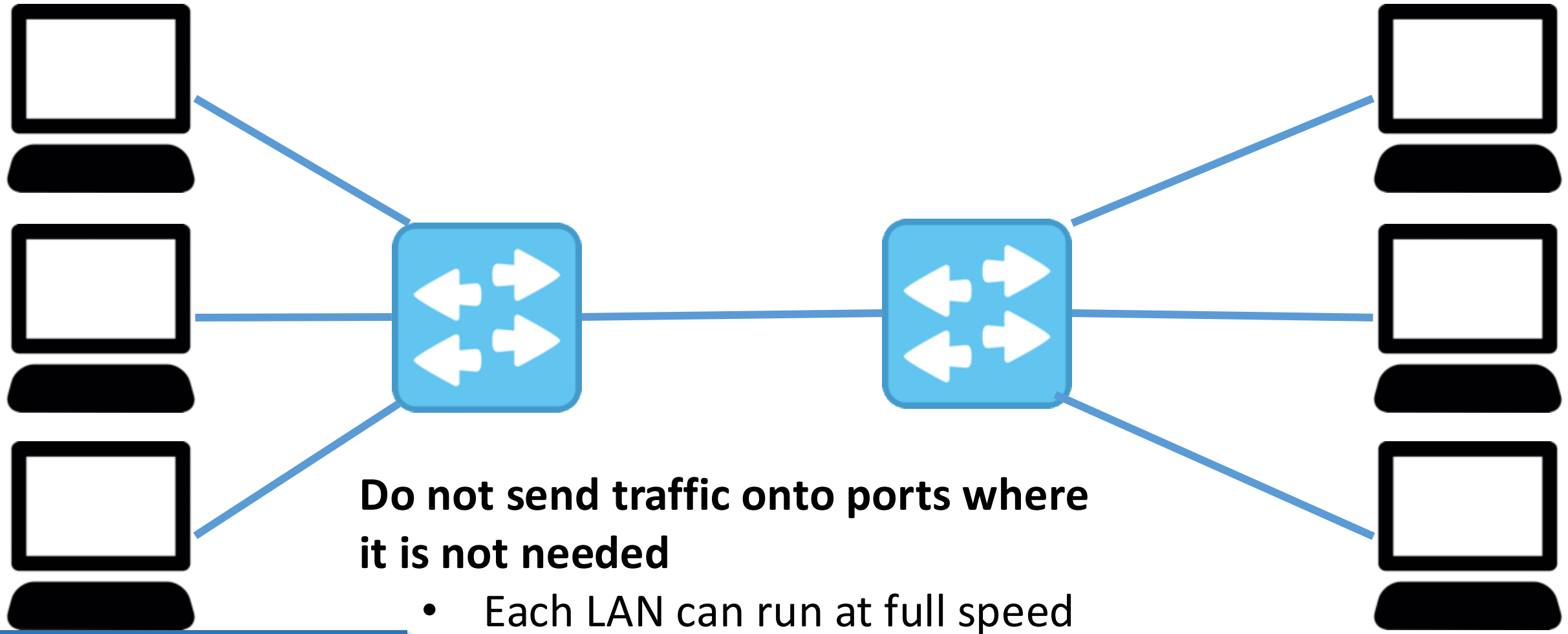


Every Layer Adds Up Their Own Services



Data Link Layer Switching

- Connect multiple LANs together



Do not send traffic onto ports where it is not needed

- Each LAN can run at full speed

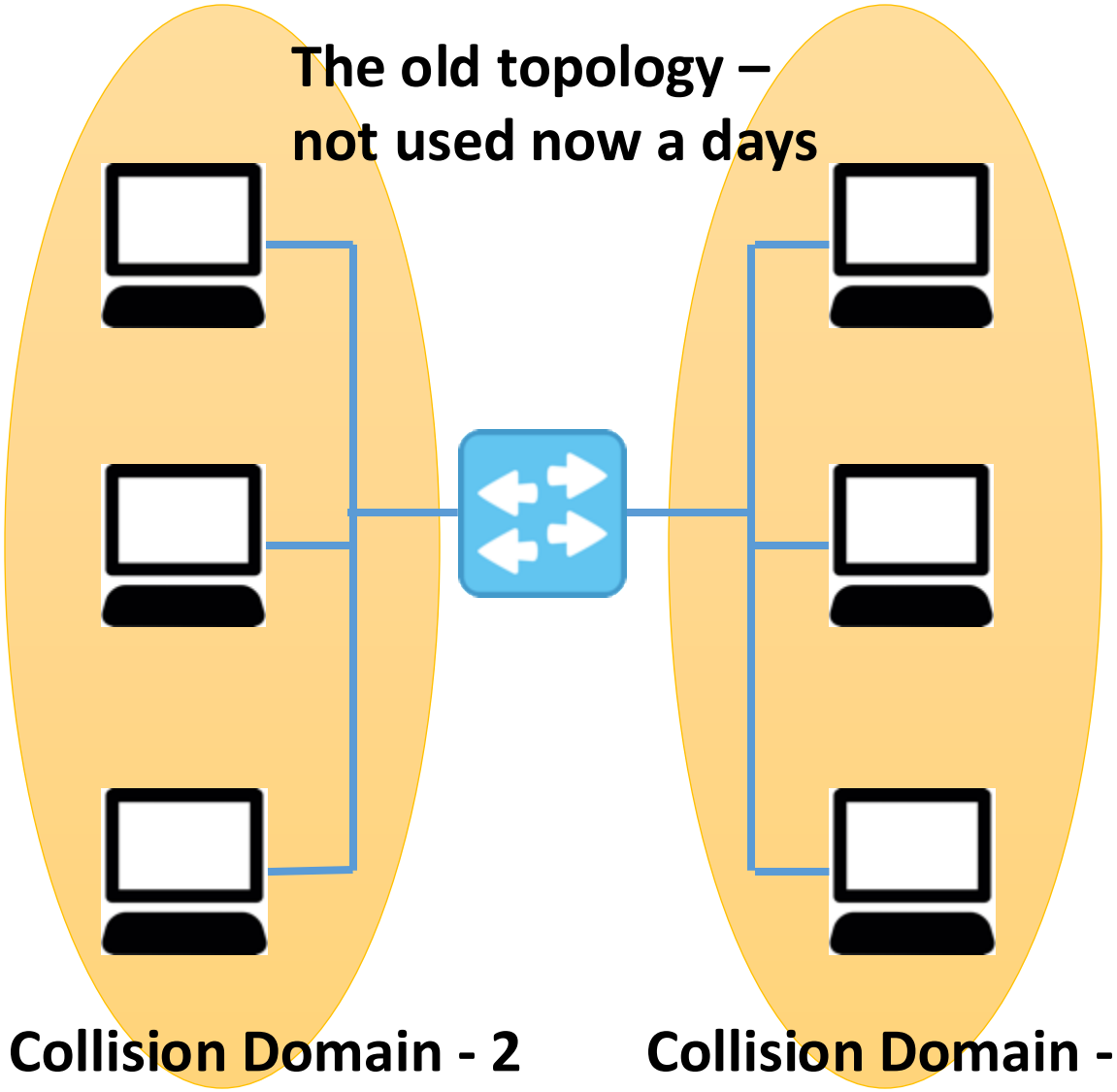
Data Link Layer Switching

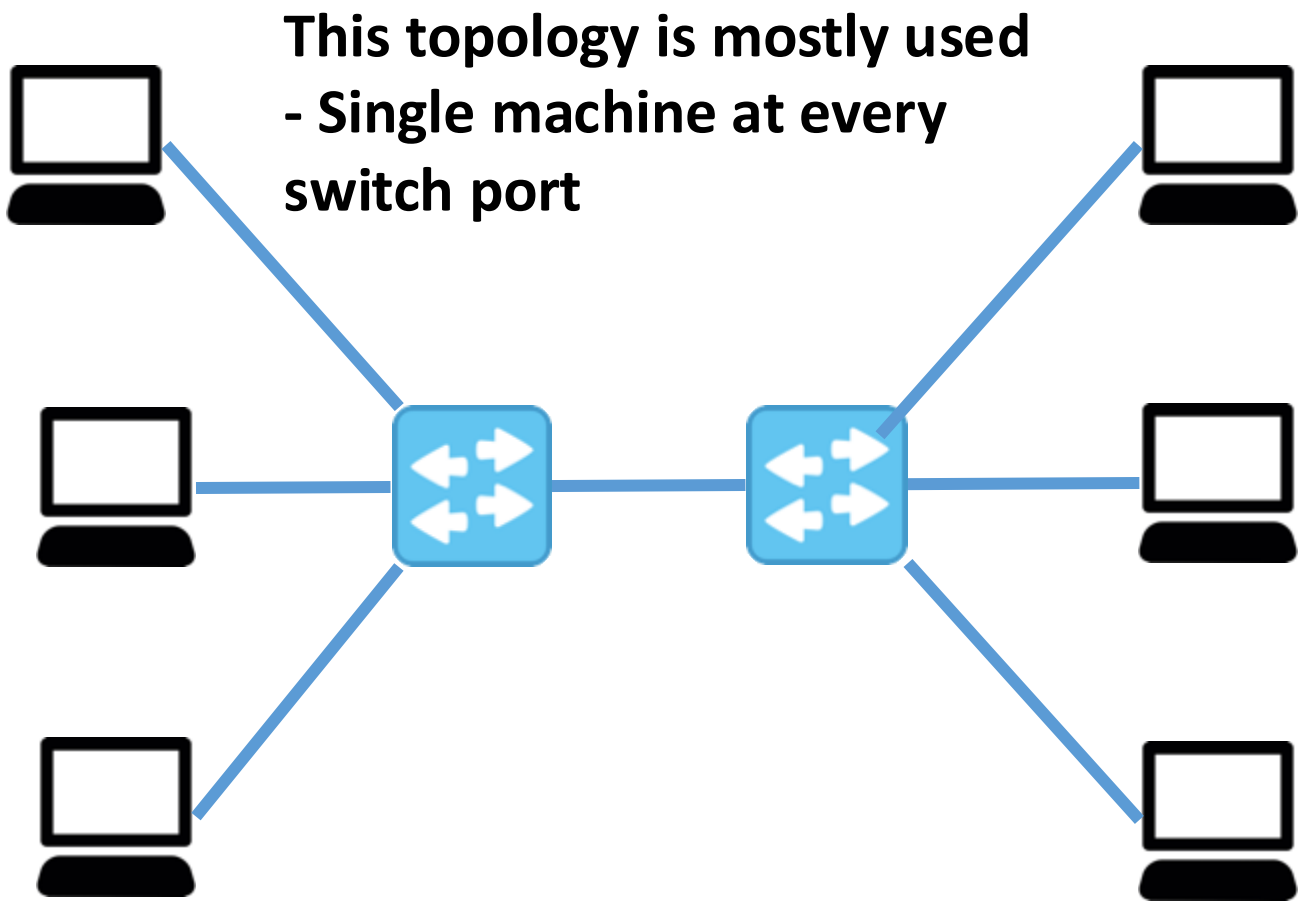
- Data link layer switches (sometime called as **bridge** – the old name) should be **completely transparent**
  - Plug and Play – no address change
  - The operations of existing LANs should not be affected
  - The functionality of the devices should remain same even if they are part of the switched LAN
  - Easy to move stations over a switched LAN similar to a normal LAN

- **Step 1**: Learning switches (backward learning algorithm) to stop traffic being sent where it is not needed

- **Step 2:** Spanning tree algorithm to break loops

Learning Switches / Bridges





Learning Switches / Bridges

- Switch operates in **promiscuous mode**
  - Accepts every frame transmitted by stations attach to each of its ports
- The switch decides whether to forward or discard each frame, and if forward which port to output the frame
- Maintain a table (Output Port, MAC address) that maps at which port what machine / machines are connected
- When the switches are first plugged in, the table is empty

Learning Switches/ Bridges

- Use **flooding** at the initialization – every incoming frame for an unknown destination is output on all the ports of the switch, except the one from where the frame has arrived

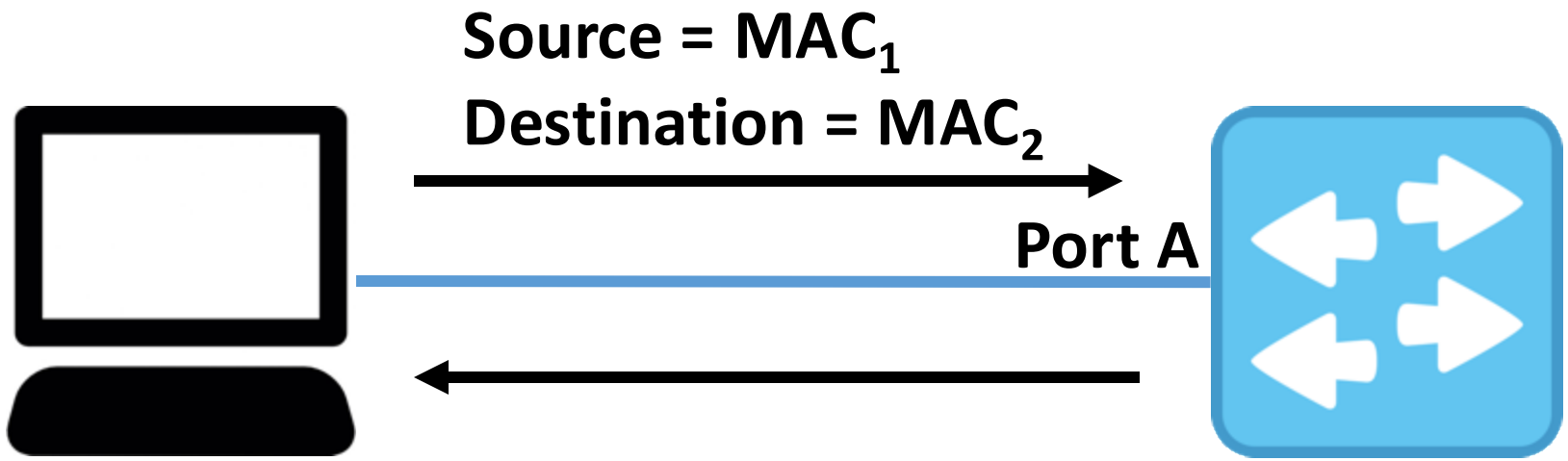
- Every network interface (machine) connected on that receives that frame
  - Decode the frame
  - Check the destination MAC address at the frame header
  - If the address matches, accept the frame
  - Else, drop the frame

- The switches starts learning about the (Output Port, MAC address) at this phase – called the **backward learning**

Learning Switches / Bridges

- For every incoming frame at a port, check the source MAC address
  - If the destination MAC is that one, then the frame needs to output at the same port

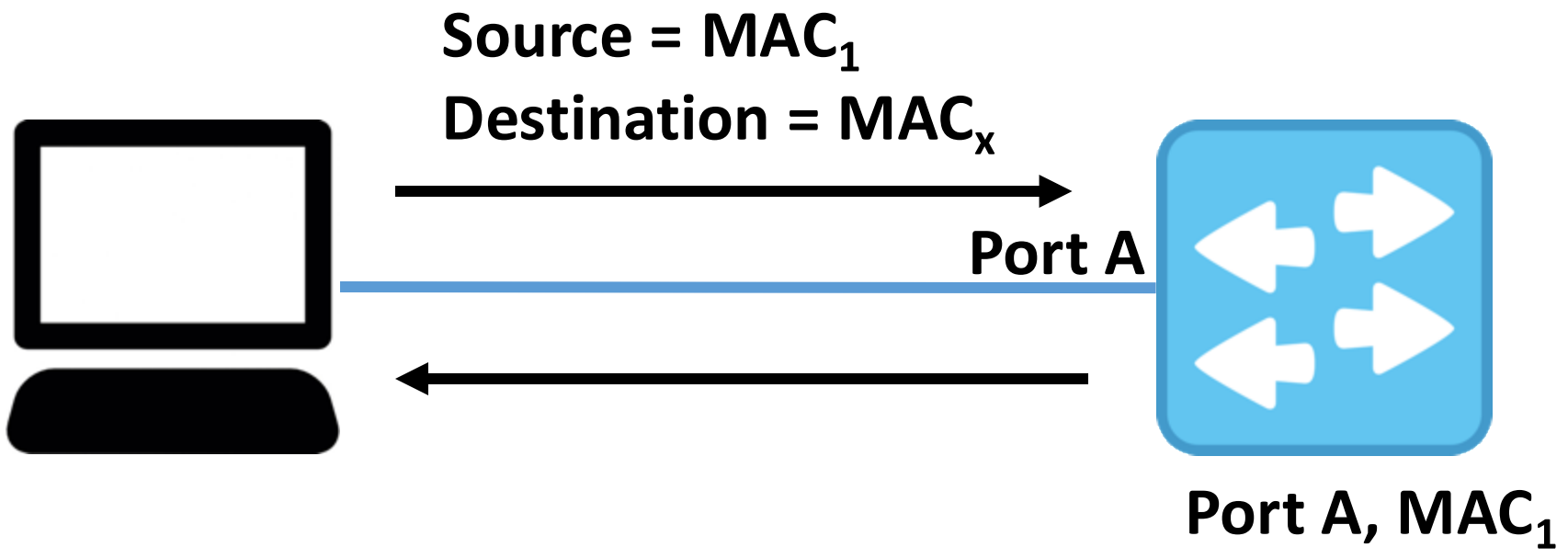
- The (Output port, MAC address) table is populated gradually by applying this backward learning



Learning Switches / Bridges

- For every incoming frame at a port, check the source MAC address
  - If the destination MAC is that one, then the frame needs to output at the same port

- The (Output port, MAC address) table is populated gradually by applying this backward learning

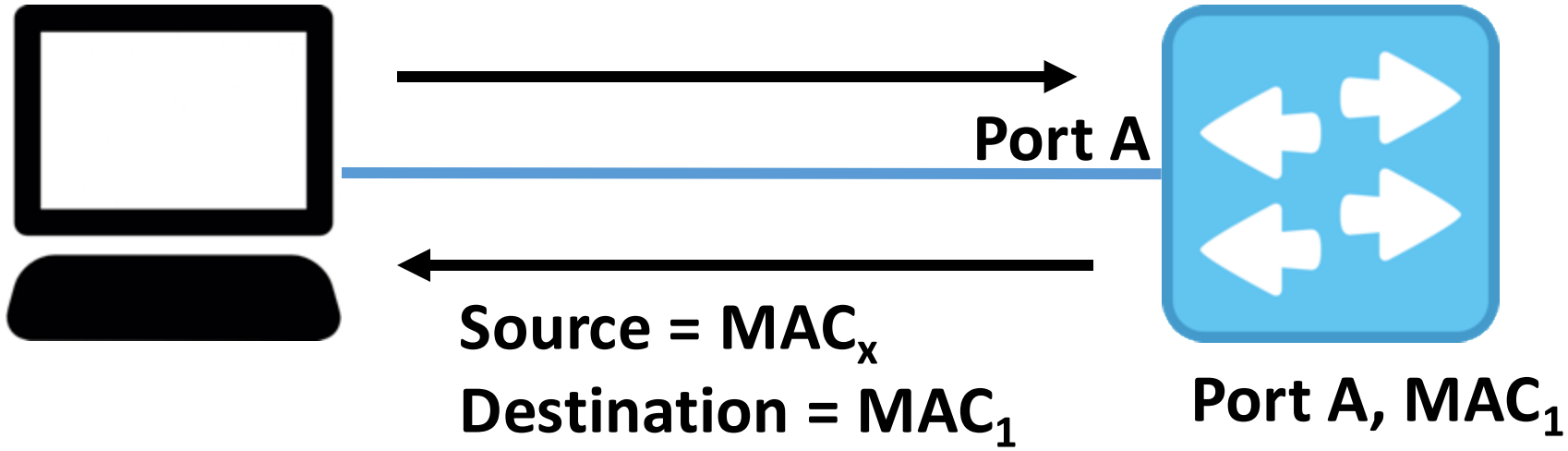


Learning Switches / Bridges

- For every incoming frame at a port, check the source MAC address
  - If the destination MAC is that one, then the frame needs to output at the same port

- The (Output port, MAC address) table is populated gradually by applying this backward learning

Delete old entries if not used for some time



Learning Switches / Bridges – The Rules / Mechanism

1. If the port for the destination address is the same as the source port, discard the frame

2. If the port for the destination address and the source port are different, forward the frame on the destination port

3. If the destination port is unknown, use flooding and send the frame on all ports except the source port

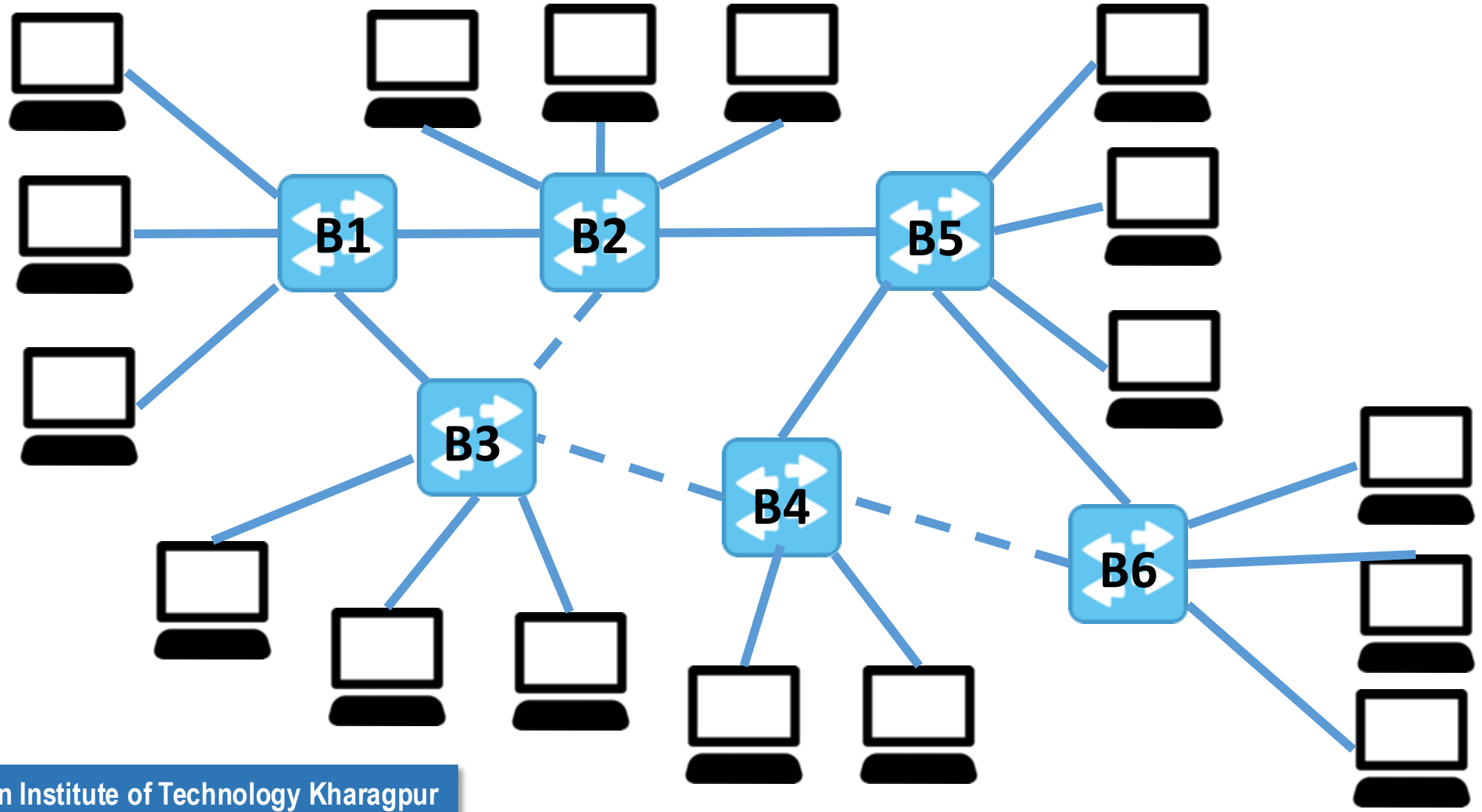
Source: Computer Network, Tanenbaum, Wetherall, The Medium Access Control Sublayer, Section 8

Cut Through Switching

- The procedure needs to be very fast - switches are technically observing all the frames coming through
- Implement in hardware
- Once you start getting the bit stream, decode them immediately based on MAC frame structure
  - Observe the source MAC and destination MAC
  - Based on the destination MAC, if the switch does not need to forward the frame, do not process it further
- Also known as **wormhole routing**

Spanning Tree Switches

- Construct a **spanning tree** across the switches



Spanning Tree Protocol (STP)

- Switches run a distributed algorithm to build the spanning tree
- Each switch periodically broadcast a **configuration message** out all of its ports, processes the message it receives from other switches
  - Embedded in STP frames
- To choose the root of the spanning tree, the switches include an identifier based on the MAC address in the configuration message, as well as the identifier they believe to be the root
- The bridges choose the bridge with the lowest identifier to be the root

Spanning Tree Protocol (STP)

- A tree of shortest paths from the root to every switch is constructed
  - Include the distance from the root in the configuration messages
- Each switch remembers the shortest path it finds to the root
- The switches then turn off ports that are not part of the shortest path
- The algorithm continues to run to detect any changes in the topology and update the tree

Spanning Tree Protocol (STP)

- Invented by Radia Perlman, 1985 (sometime called as “Mother of Internet”)

“I think that I shall never see
A graph more lovely than a tree.
A tree whose crucial property
Is loop-free connectivity.
A tree that must be sure to span
So packets can reach every LAN.
First, the root must be selected.
By ID, it is elected.
Least-cost paths from root are traced.
In the tree, these paths are placed.
A mesh is made by folks like me,
Then bridges find a spanning tree.”

-Radia Perlman

Network Devices at Different Layers

Application Gateway	Application Layer
Transport Gateway	Transport Layer
Router	Network Layer
Bridge, Switch	Data Link Layer
Repeater, Hub	Physical Layer

Error Detection and Correction

- Bit errors can be introduced during transmission of the signal – more common for wireless networks
- Add redundant information with the frame that is sent
- **Error detection:** Include enough redundancy so that the receiver can separate out a correct frame and an erroneous frame (cannot identify the actual error)
- **Error correction:** Include enough redundancy so that the receiver can correct errors up to some level (detect which bits are having an error)

Error Detecting Codes - Parity Bit

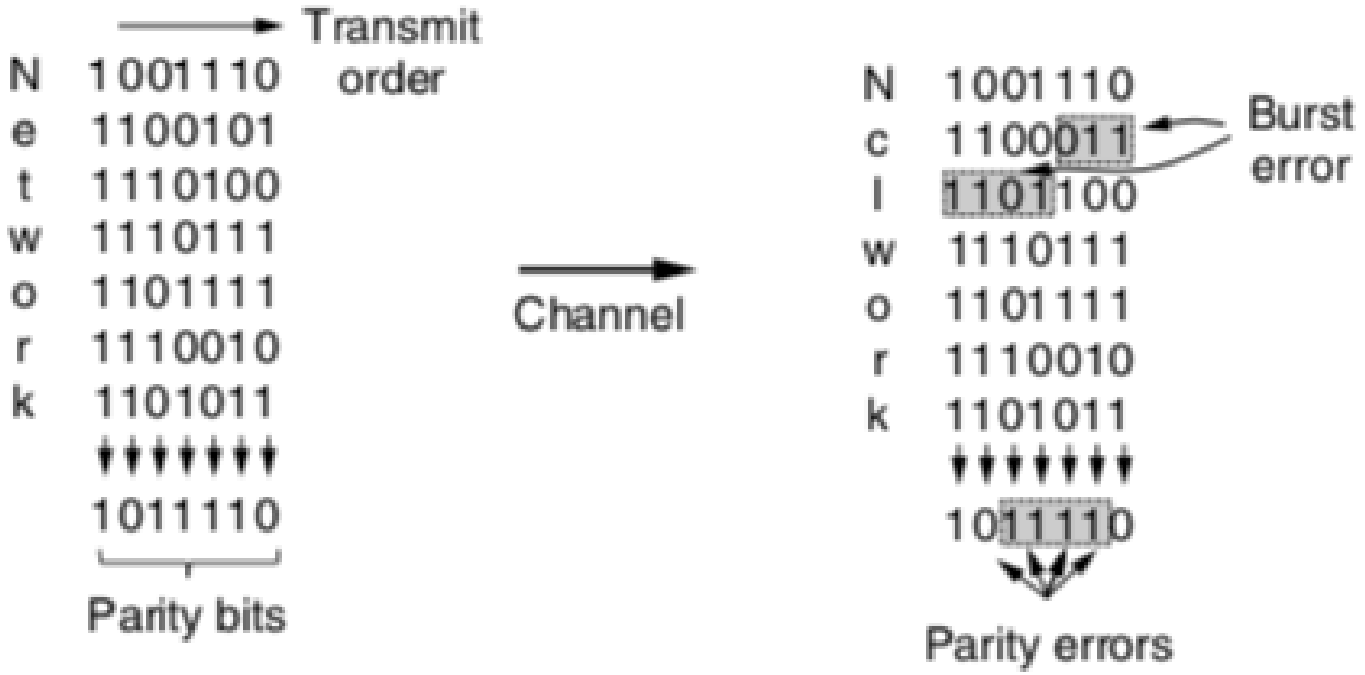
- Include a single parity bit with the data.

- The parity bit is chosen so that the number of 1 bits in the codeword is even (or odd, depending on the setup)
  - 1101110011 is the data to transmit
  - Codeword is 11011100111 for even parity transmission

- Can detect an odd numbers of bit errors – single bit error can be reliably detected
  - Difficult to detect for burst error

Error Detecting Codes – Interleaving Parity

Even parity



N parity bits are used for kN data bits
k bit errors will be reliably detected with at most one error per row

Source: Computer Network, Tanenbaum, Wetherall, The Data Link Layer, Section 2

Error Detecting Codes – Checksum

- Let us look into the case for 16-bit Internet checksum
- Divide the data bits into 16 bit words
- Use 1’s complement arithmetic to compute the sum
- Find out 1’s complement of the result - that’s the checksum

Example – 8 bit Checksum

- Say, we want to transmit 1101110011001110
- Two 8 bit words – 11011100 and 11001110
- 1's complement addition - 00111100
- 1's complement of the results – 11000011 – **that's the checksum**
- More rigorous than parity check – two bit flipped remains undetected in parity check, but may be detected in checksum

Error Detection Code – Cyclic Redundancy Check (CRC)

- Also known as **polynomial code** – treat bit stream as a representation of polynomials with coefficient 0 and 1

- For example, 1011001 can be treated as $1.x^6 + 0.x^5 + 1.x^4 + 1.x^3 + 0.x^2 + 0.x^1 + 1.x^0$, which is equivalent to $x^6 + x^4 + x^3 + 1$

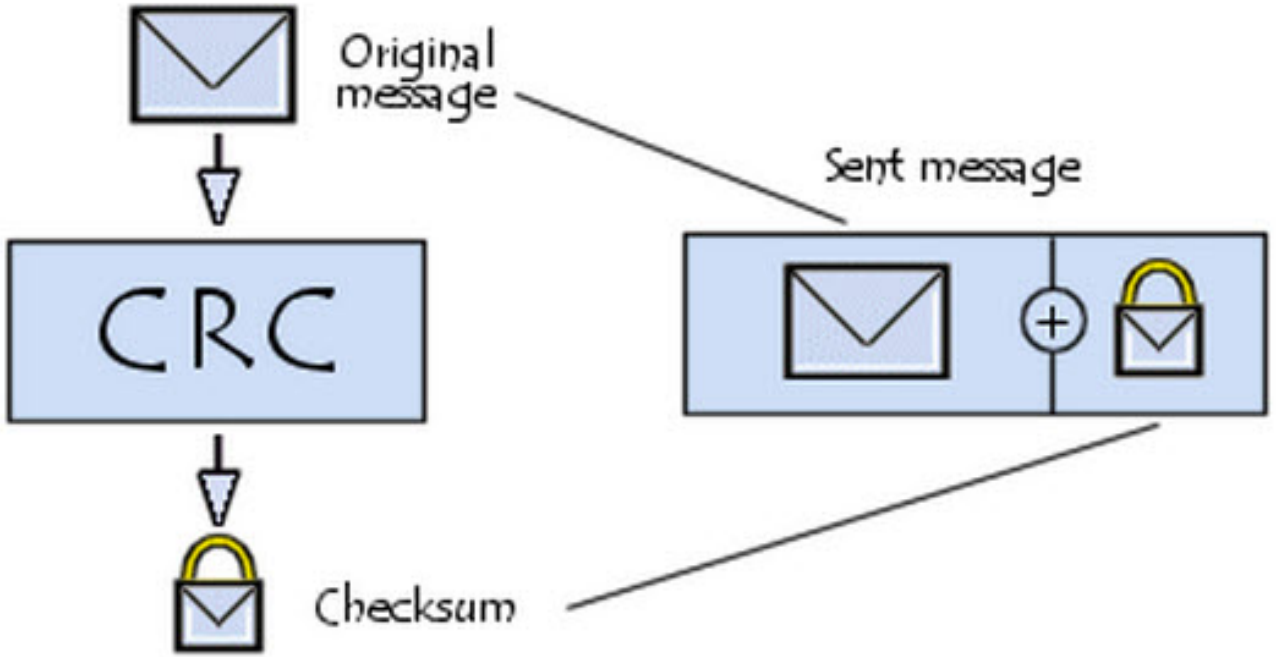
- The sender and the receiver must agree upon a **generator polynomial** G(x)
  - Both the high and the low order bits for the generator polynomial must be 1

- To compute CRC for a polynomial M(x) corresponds to a frame, the number of bits in the frame must be longer than the generator polynomial

Error Detection Code – Cyclic Redundancy Check (CRC)

- Append CRC at the end of the frame, such that the polynomial represented by the CRC included frame must be divisible by G(x)

- The receiver checks whether the received frame is divisible by G(x), if no, then there has been a transmission error



Error Detection Code – Cyclic Redundancy Check (CRC)

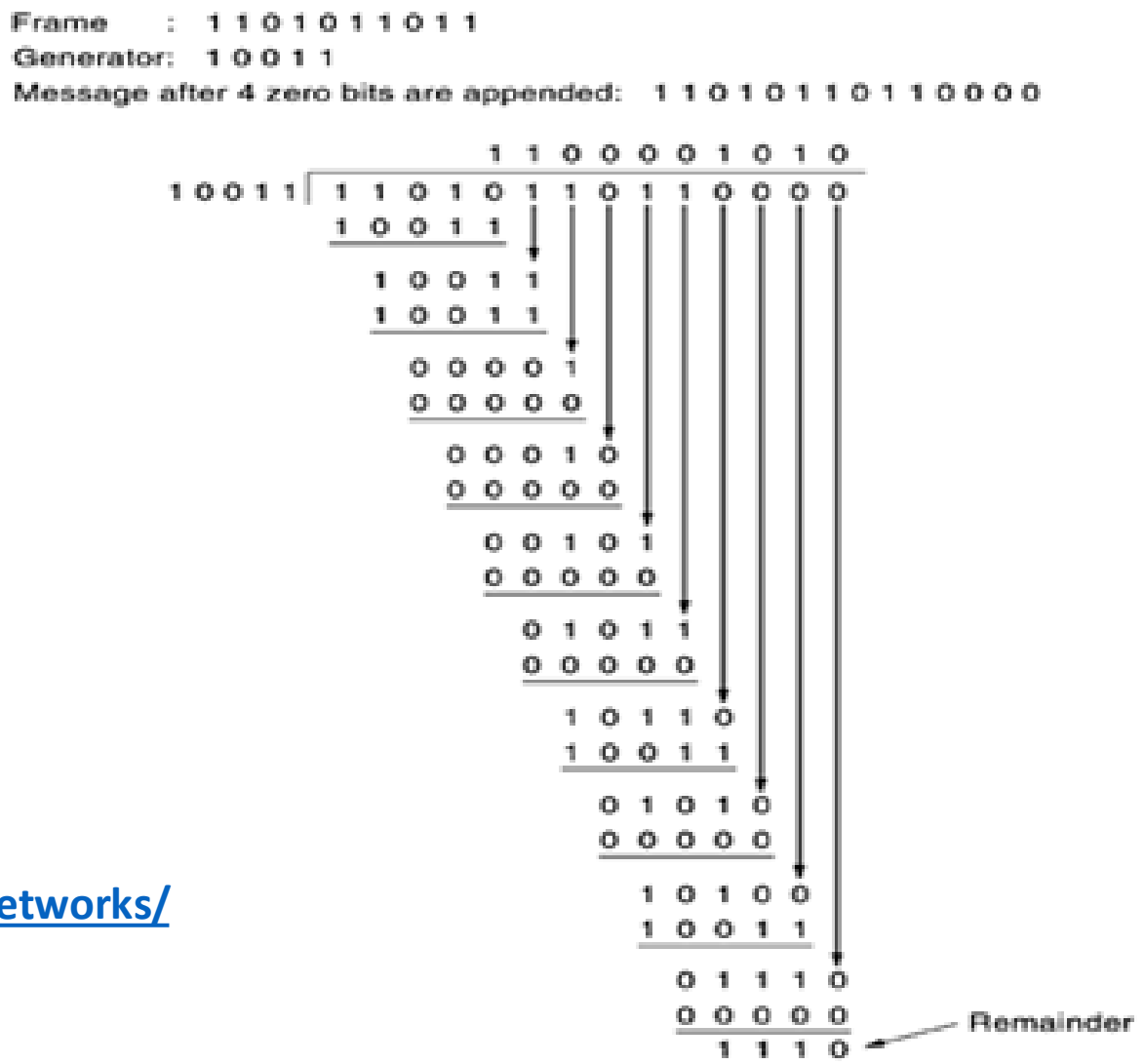
1. Let r be the degree of G(x), append r zero bits to the low order end of the frame, so the frame contains m+r bits where m is the number of bits in the original frame. The polynomial corresponds to $x^rM(x)$

2. Divide the bit string corresponds to $x^rM(x)$ by the bit string corresponding to G(x) using modulo 2 division

3. Subtract the remainder (which is always r bits or fewer) from the bit string corresponding to $x^rM(x)$ using modulo 2 subtraction. The result is the checksum frame to be transmitted
   - Subtraction is equivalent to XOR in modulo 2 arithmetic

CRC – Example

A polynomial code with r check bits can detect all burst errors less than on equal to r



Source:
http://computing.dcu.ie/~humphrys/Notes/Networks/data.polynomial.html

Forward Error Correction (FEC)

- Let a frame consists of m data bits and r check bits
- **Block codes:** The r check bits are computed solely as a function of the m data bits
- **Linear codes:** The r check bits are computed as a linear function of the m data bits, example: XOR with modulo 2 arithmetic
- Error correcting codes – the r check bits that are sent with the data bits for correcting a single or multiple bit errors during the transmission

Hamming Distance

- The number of bit positions in which two codeword differs (Hamming, 1950)

- Example

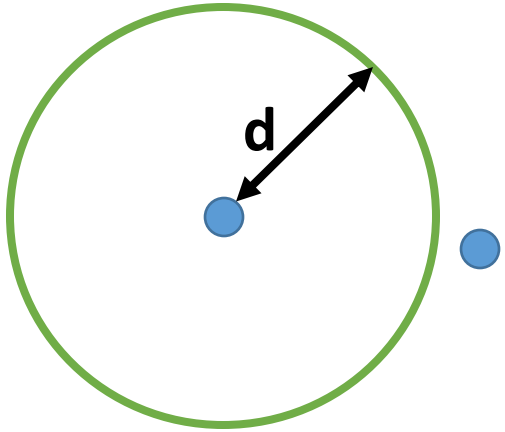
Codeword-1	11001101
Codeword-2	01001010
XOR	10000111

Hamming distance is 4

- If two codewords are a Hamming distance d apart, it will require d single bit errors to convert one into another

Hamming Distance for Error Detection

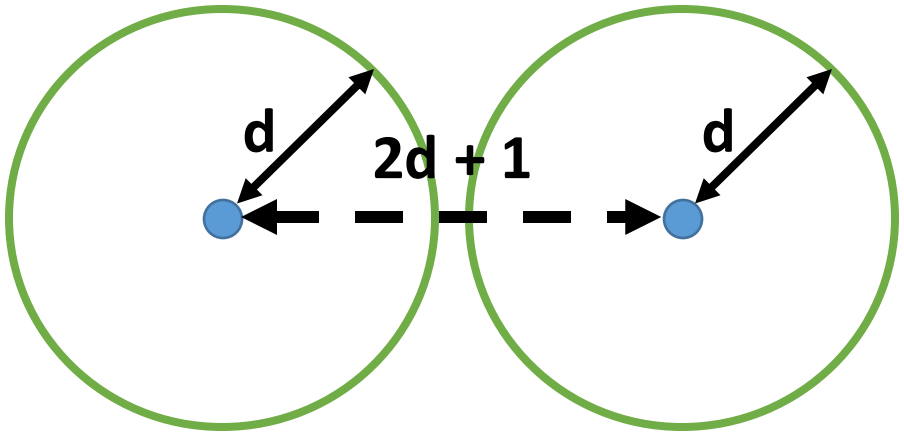
- Number of possible m bit data messages 2^m
- Number of possible m+r codewords 2^{m+r} – all the codewords are not valid codewords
  - We can have 2^m different codewords corresponding to 2^m different messages
- To **detect** d bit errors, we need to ensure that the Hamming distance between two valid codewords is d+1



Hamming Distance for Error Correction

- We need to map every invalid codeword to a valid codeword

- To correct d single-bit errors, the minimum Hamming distance between two codewords should be at least 2d+1

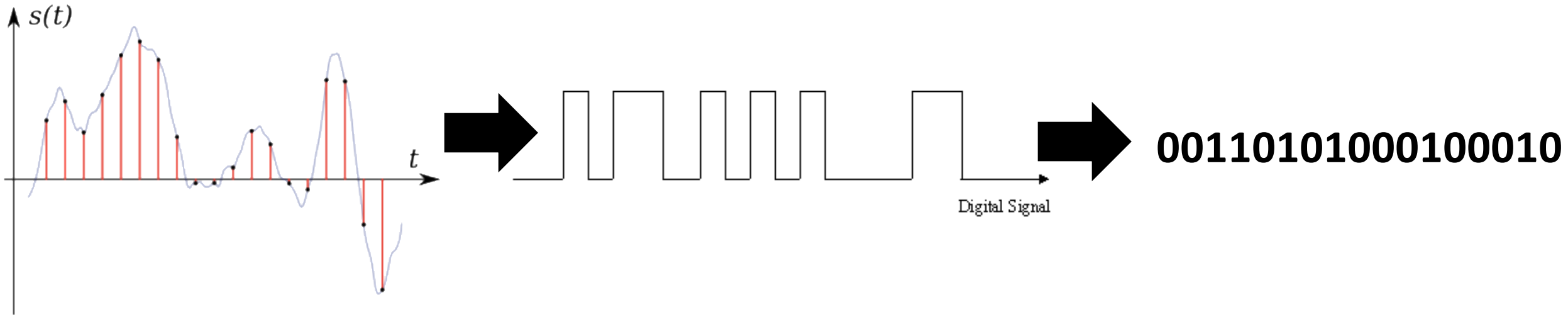


Hamming Distance for Error Correction - Example

- Consider the four codewords
  - 0000000000
  - 0000011111
  - 1111100000
  - 1111111111
- The hamming distance is 5 – we can correct single and double bit errors
- If the received codeword is 1010100000, we can expect the original codeword be 1111100000
- Say you have received 1010000000, and there are maximum three bit errors, guess the original codeword

Framing

- Identify a frame from the received bits



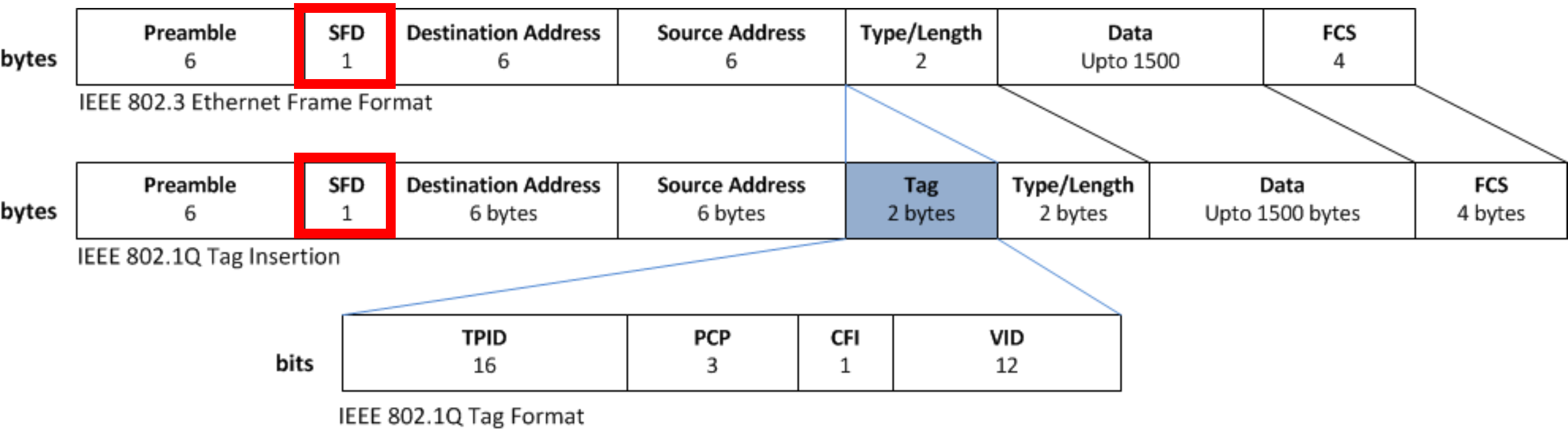
- How do you you identify which part of the bit stream is a single frame?

- Can you use a length field in the header?

Flag bits at the Beginning of the Frame

- Start frame delimiter (SFD) and end frame delimiter (EFD)
- A special bit sequence indicating beginning of a frame
- EFD is deprecated and is not used now a days
- Ethernet SFD: 10101011
- Preamble: Synchronize the sender and the receiver – sequence of 10101010

Ethernet Frame Structure

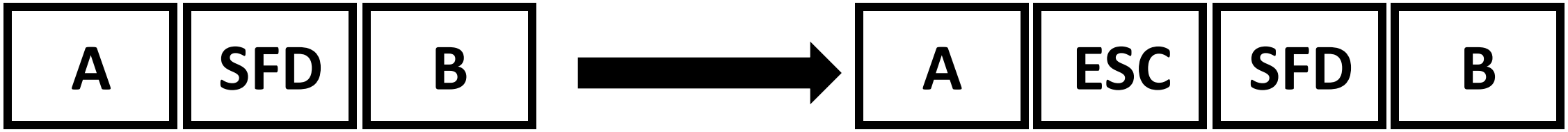


TPID = Tag Protocol Identifier
PCP = Priority Code Point
CFI = Canonical Format Indicator
VID = VLAN Identifies (VLAN ID)

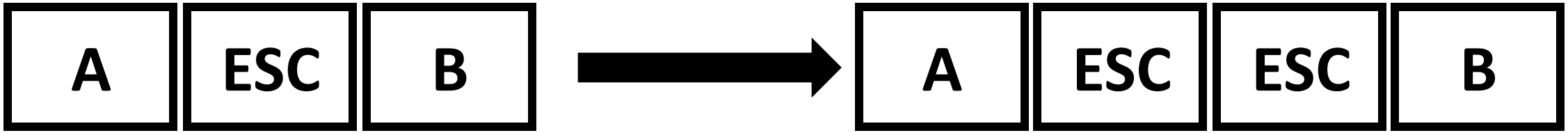
Byte Stuffing (PPPoE)

- 10101011 can also be a part of the payload

- **Solution:** Insert a special escape byte (ESC) if the SFD is part of the data – used in Point to Point Protocol (PPP)



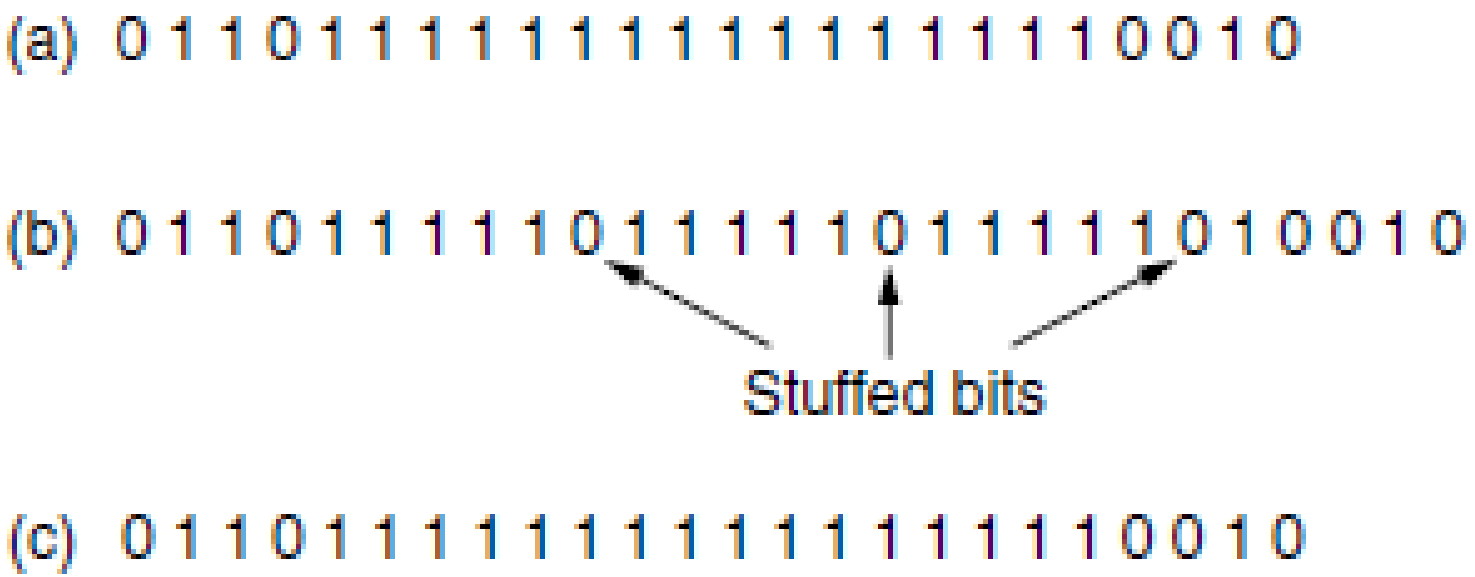
- **ESC can be part of the data as well** – insert another ESC before the ESC



Bit Stuffing (HDLC)

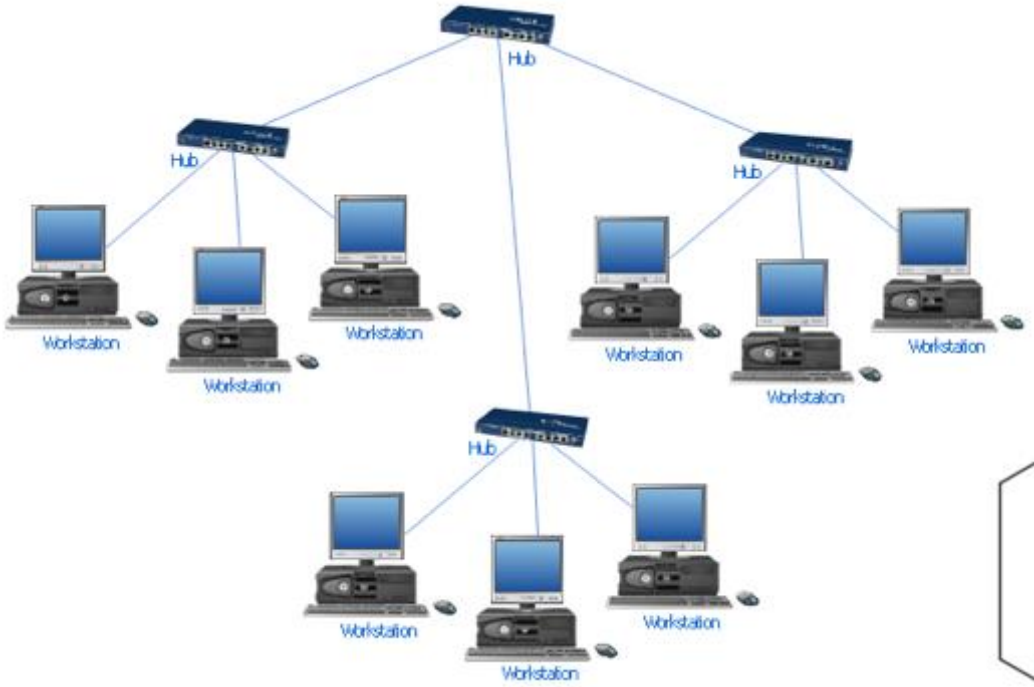
- Byte stuffing has the overhead of one byte per ESC character
- Use bit stuffing instead of byte stuffing – implemented in **High-level Data Link Control (HDLC) protocol**
- In HDLC, each frame begins and ends with 01111110 (called FLAG byte)
- Six consecutive 1's in the data can get confused with this FLAG byte
- Stuff a 0 bit if there is 5 consecutive 1's in the data bit – transmit 011111010 instead of 01111110

Bit Stuffing Example



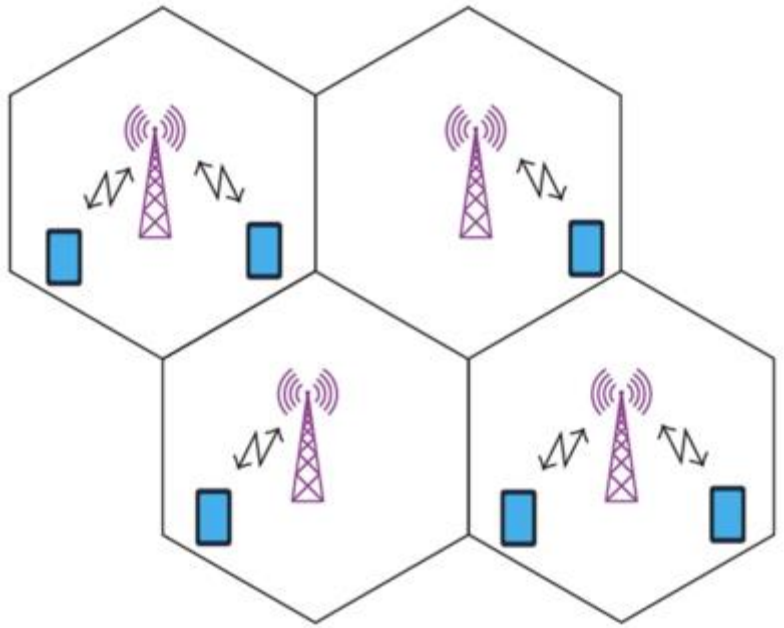
The Channel Allocation Problem

- Many of the times, the media is a broadcast media



Devices are connected to a hub

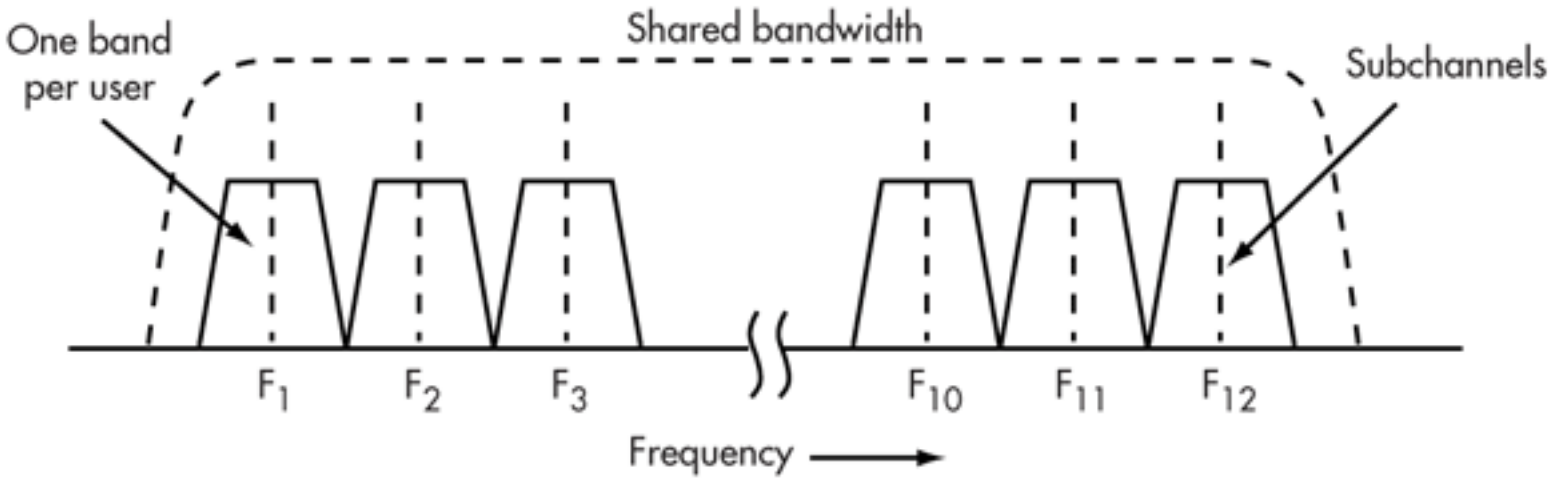
A wireless Network



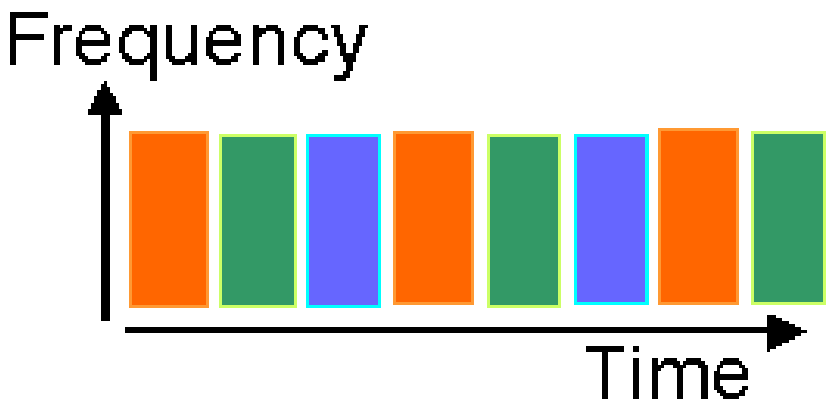
A Cellular Network

Static Channel Allocation

- **Frequency Division Multiple Access (FDMA):**



- **Time Division Multiple Access (TDMA):**



Dynamic Channel Allocation

- Static Channel Allocation has multiple problems:
  - Resource wastage when the user is idle
  - Strong synchronization is necessary
- Dynamically allocate channels to the end users whenever required – multiple access protocols
- Probabilistic scheduling among devices or users - contention based systems
  - Schedule the nodes such that contention can be minimized

Multiple Access Protocol

- **Pure Aloha:** Transmit the frame whenever you have one
  - Results in collision
  - The hub rebroadcast the frame (works as an acknowledgement)
  - If the sender does not hear its frame within a timeout, retransmission the frame again

- **Slotted Aloha:** Divide time into discrete interval called slots. Transmit the frame at the beginning of the next slot
  - The users need to agree on slot boundaries
  - Collision can still happen, retransmit after timeout
  - Doubles up the efficiency compared to pure Aloha – check the Tanenbaum book for details !

- Best channel utilization with slotted aloha is 1/e (e=2.71828), approx. 36.7%

Carrier Sense Multiple Access (CSMA)

- **Physical carrier sensing (PCS):** Sense the carrier to see whether there is any ongoing signal.
- If PCS returns the channel to be idle, the station sends data
- Otherwise wait until the channel becomes idle
- If collision occurs, wait for random amount of time and repeat the process
- We call this as **1-persistent CSMA**

p-persistent CSMA

- Divide the time to logical slots
- When station is ready to send a frame, sense the channel (PCS)
- If the channel is idle at the current slot, then transmit with probability *p*
- With probability *1-p*, target the next slot, and repeat the process
- Non-persistent CSMA: If the channel is sensed busy through PCS, wait random amount of time, and then sense again

Collision in CSMA (CSMA/CD)

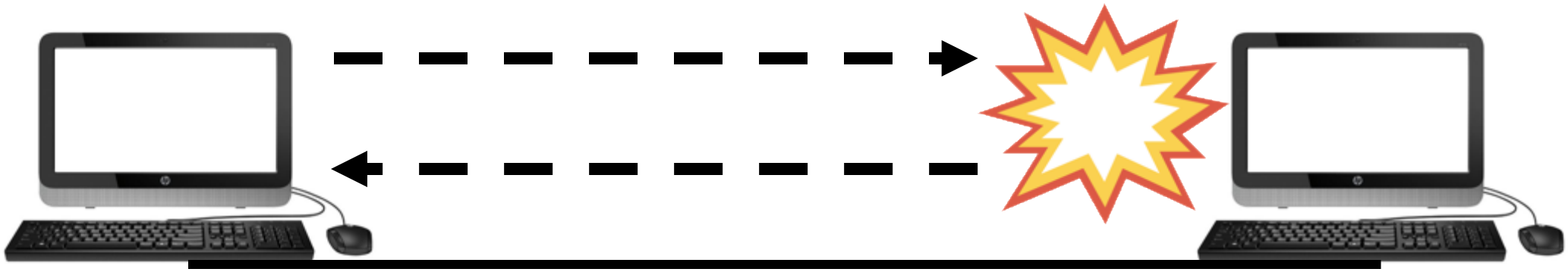
- Two stations may sense the channel idle at the same time, and starts transmission

- A collision can happen in the channel – detect the collision
  - Collision signal will differ from the normal signal

- **Random Back-off:** Wait for random amount of time if a collision is detected

- **Binary Exponential Back-off:** At n^{th} attempt, wait for time equivalent of random$(0,2^n-1)$ slots

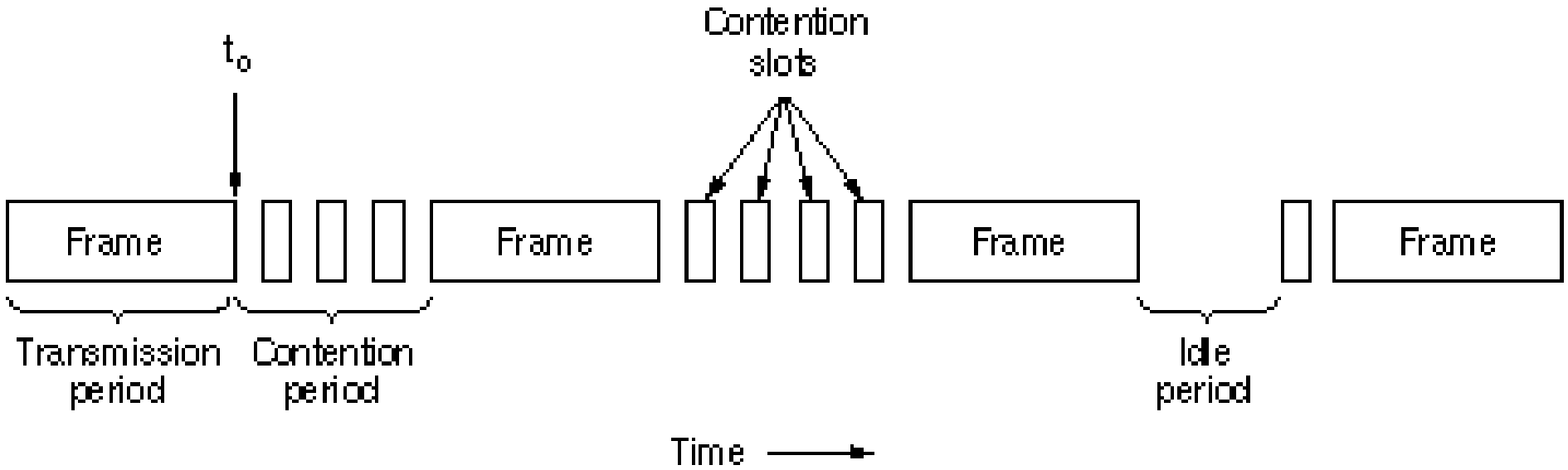
Collision Detection and its Impact over MAC Frame Length



- Worst case collision detection time = 2 x propagation delay

- To detect a collision, the minimum frame transmission time should be equal to 2 x propagation delay (eg. Minimum Ethernet frame size = 64 bytes)

Ethernet Performance - Contention Interval



- Contention interval is the time when the channel bandwidth is not utilized due to collision

- We need to estimate the probabilistic length of a contention interval

Ethernet Performance

- k stations are ready to transmit. Each station transmits during a contention slot with probability p
- Consider a given slot. What is the probability A that some station acquires the channel in that slot?

$$A = kp(1-p)^{k-1}$$

- A is maximized when $p = 1/k$, with $A \to 1/e$ as $k \to \infty$
- The probability that the contention interval has exactly j slots in it is $A(1-A)^{j-1}$
- What is the mean number of slots per contention?

$$\sum_{j=0}^{\infty} jA(1-A)^{j-1} = \frac{1}{A}$$

Ethernet Performance

- Let τ be the propagation delay, a slot duration is set to 2τ to ensure collision detection within the slot duration.
- We observed that mean number of slots per contention is $\frac{1}{A}$
- Mean contention interval $w = \frac{2\tau}{A}$
- We observed that with optimal p, $A \to 1/e$, So, the maximum value of w is $2\tau e = 5.4\tau$

Ethernet Performance

- Mean frame transmission duration is P

- Channel efficiency = $\frac{P}{P+2\tau/A}$

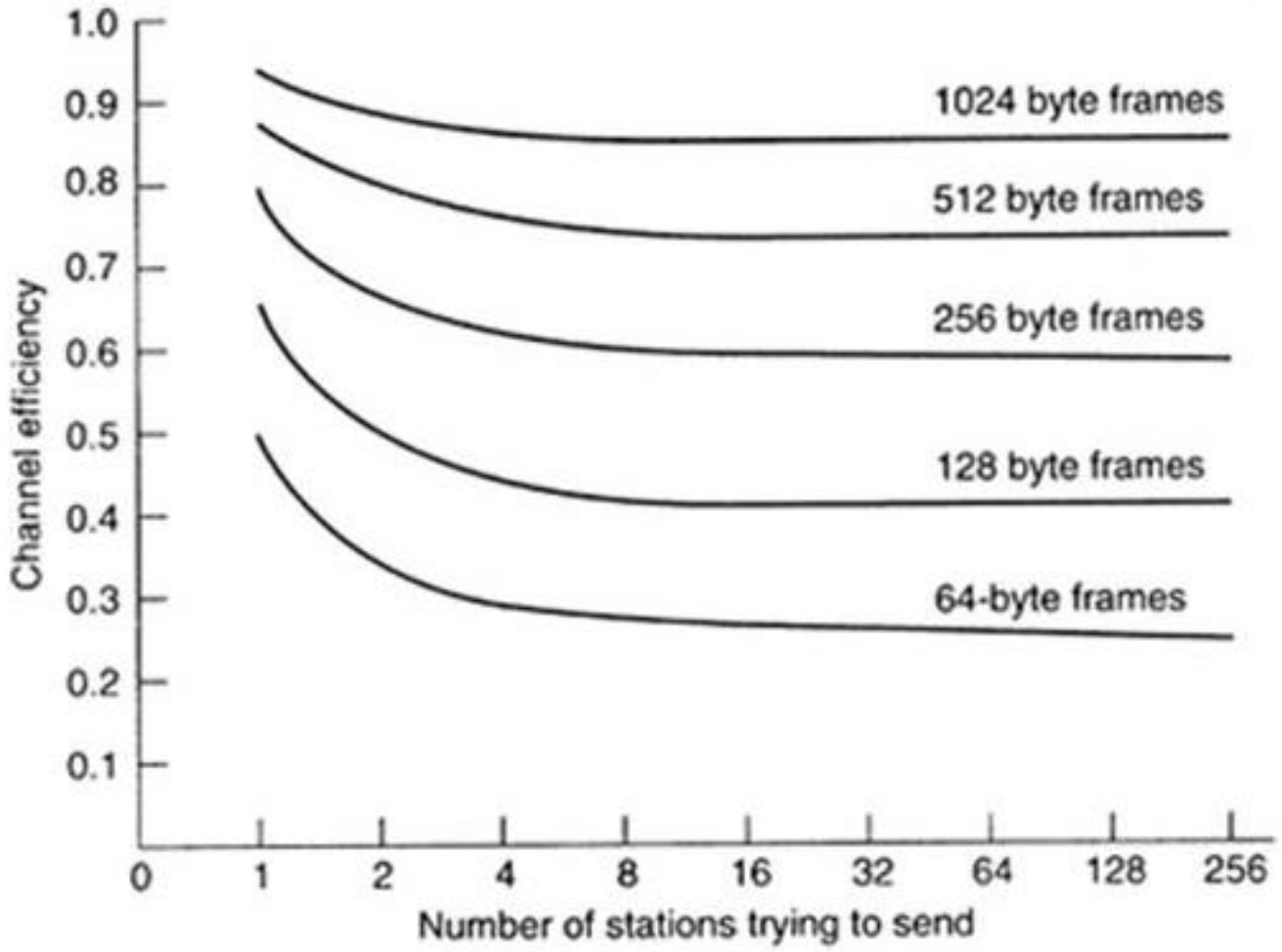


Image Source: Computer Networks, Tanenbaum

Can You Detect Collision over Wireless?

- Wireless Signal
  - **Fading –** Attenuation is high**,** Signal strength drops as the signal propagates
  - **Shadowing –** Gets affected by shadowed textures, like wall
  - **Multipath Propagation –** Reflected signals may come from different directions

- **Additive Interference**: Signals from multiple sources gets added up and interfere with the communication of a third device
  - Individual signals may not enough for interference

- Interference range is different from the transmission range

Hidden Terminal Problem

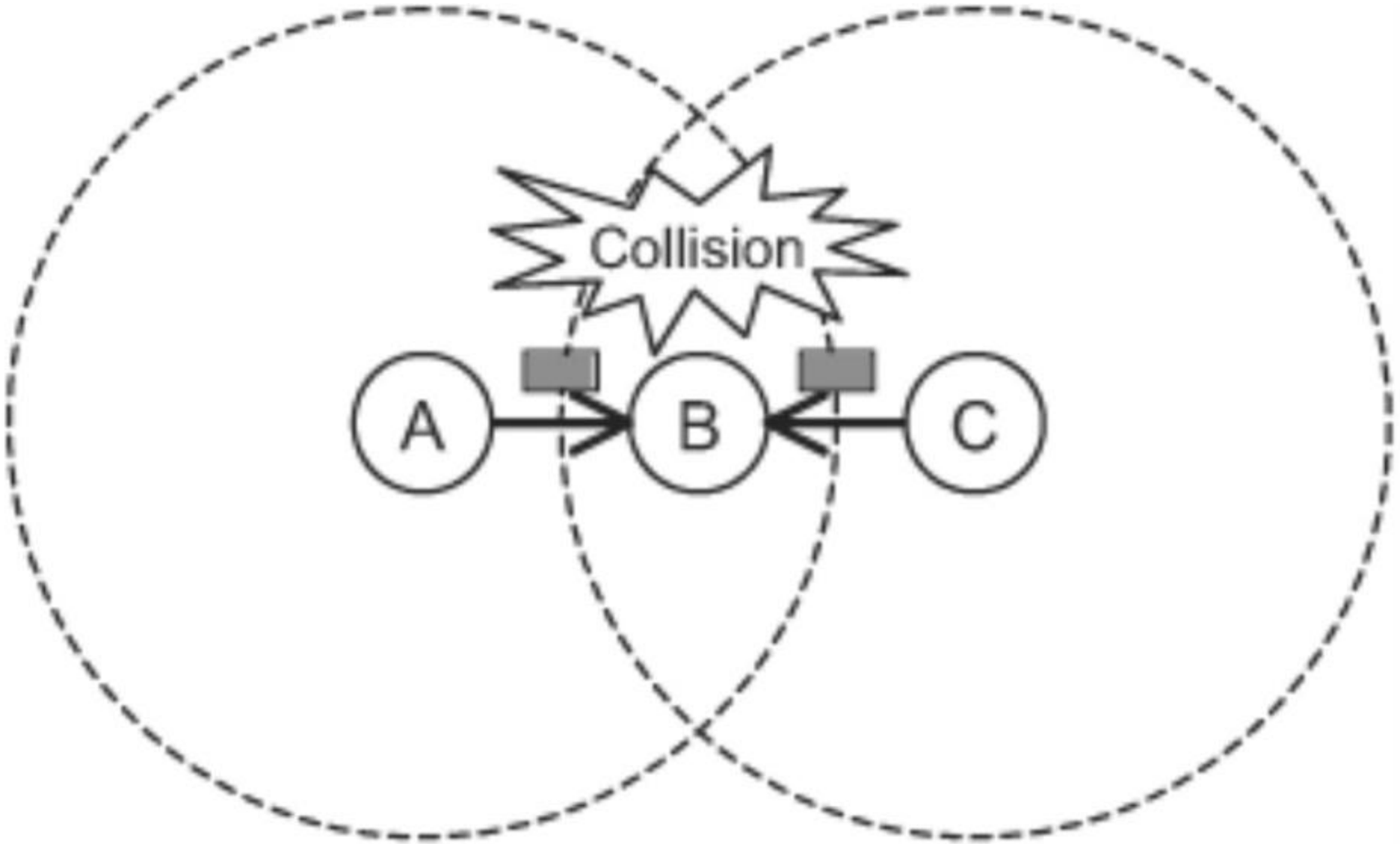


Image Source: https://www.youtube.com/watch?v=X-H2tLesBUQ

Exposed Terminal Problem

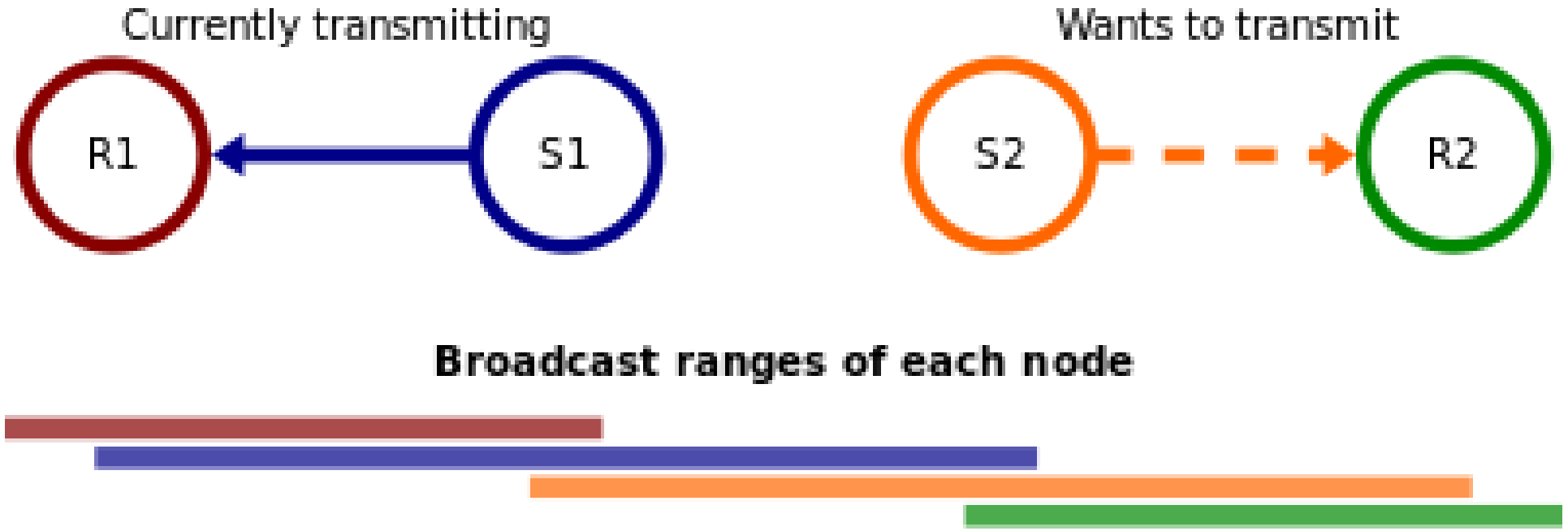


Image Source: https://en.wikipedia.org/wiki/Exposed_node_problem

Wireless Scenario

- Collision detection is not feasible for wireless
- How will detect a data loss due to interference?
  - Sender transmits a DATA
  - Receiver sends an ACK
- Assume the frame to be lost, if ACK is not received within a predefined timeout.

Virtual Carrier Sensing (VCS) – CSMA/CA

Request to Send (RTS)
Clear to Send (CTS)

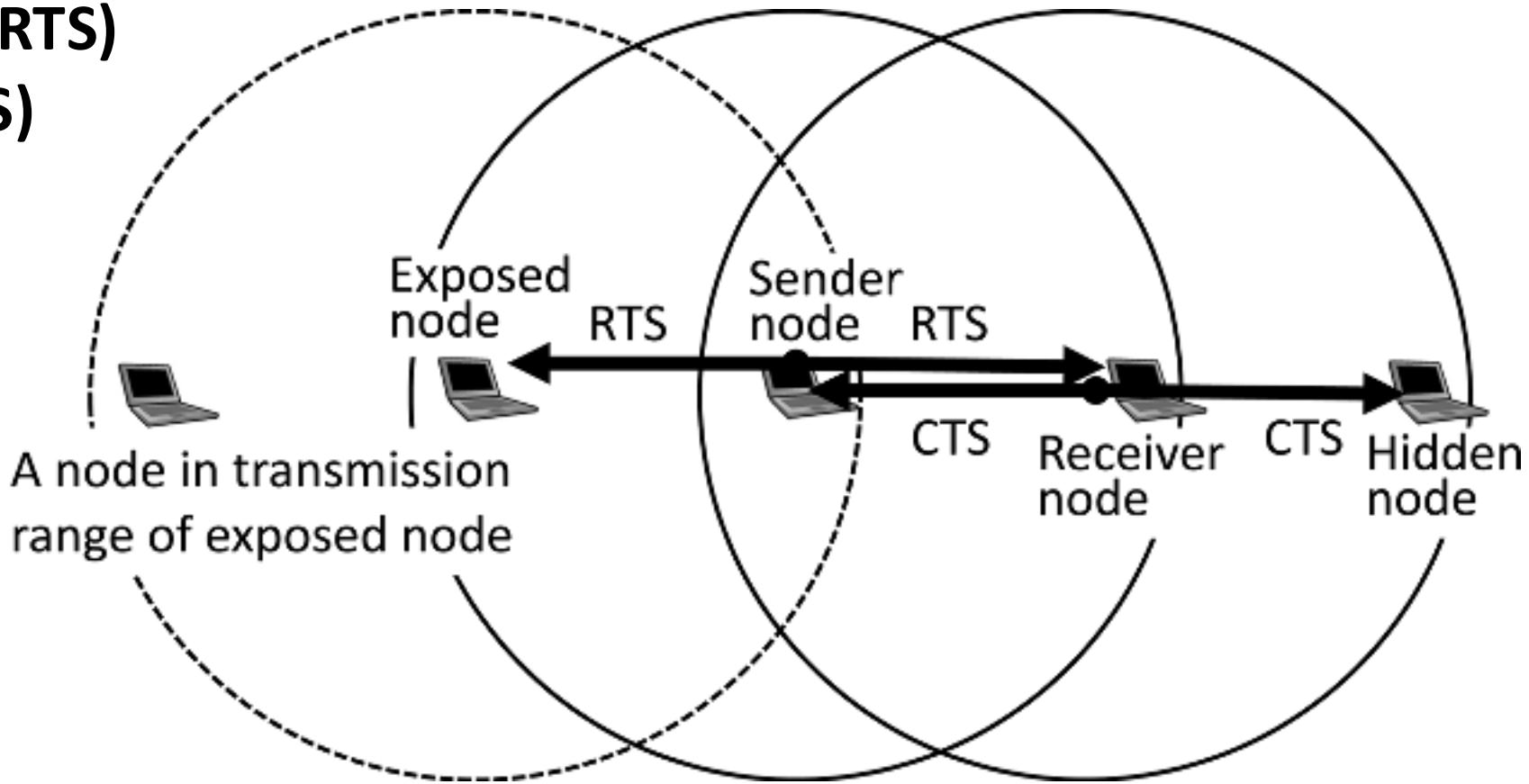


Image Source: Asymmetric RTS/CTS for Exposed Node Reduction in IEEE 802.11 Ad Hoc Networks

Setting up Network Allocation Vector (NAV)

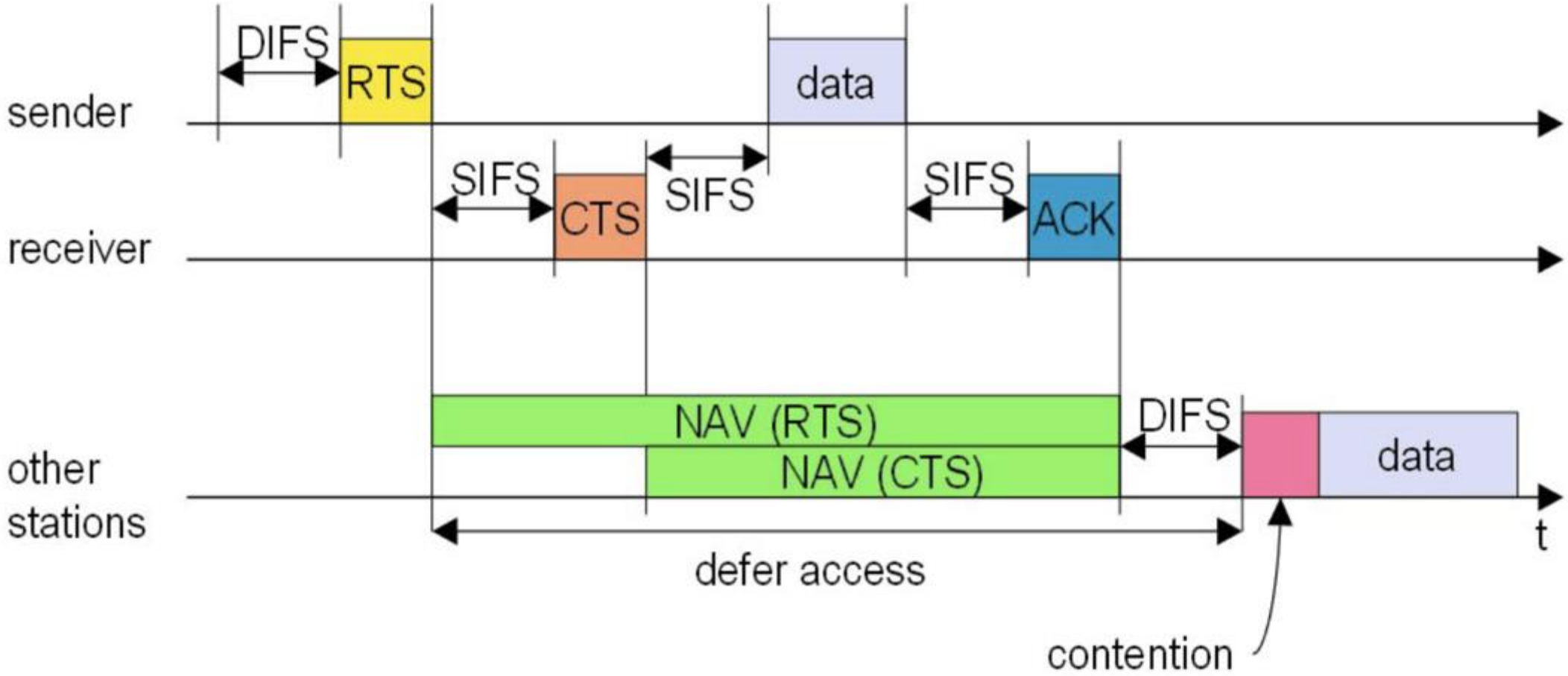


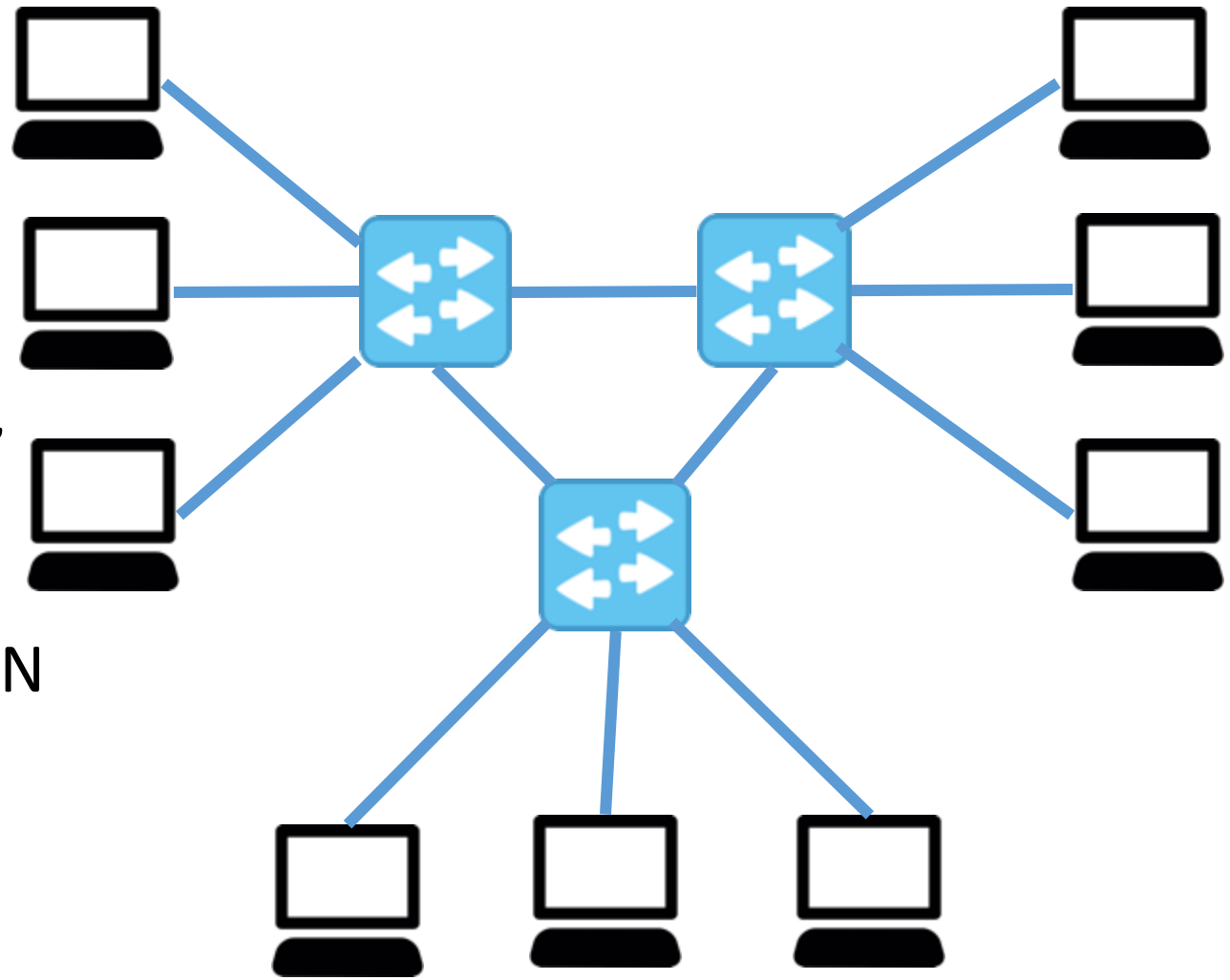
Image Source: https://community.arubanetworks.com/t5/Technology-Blog/Understanding-802-11-Medium-Contention/ba-p/232034

IEEE 802.11 Distributed Coordination Function (DCF)

1. Set n = 0 */*n denotes the round number*/*
2. Use PCS (and VCS) to sense the channel
3. If the channel is busy, repeat 2
4. If the channel is idle for DIFS duration, transmit the frame, set timeout
5. If ACK received within timeout, Go to 1
6. If timeout, set $CW_{min} = min (2xCW_{min}, CW_{max})$
7. Set n = n+1
8. Set $CW = random (0, CW_{min})$
9. Wait for CW number of slots and go to 2

Spanning Tree Switches

- To increase reliability, redundant links may be deployed between the switches
- Creates loop in the topology
- Note that for unknown destination, switches flood the frames
- Such frames can loop within the LAN



Virtual LAN (VLAN)

- LAN is a **broadcast domain**
  - Every frame is a broadcast to all the machines connected in a switch port

- **VLAN**
  - Construct a broadcast domain even if they are not a part of the switch port – use **VLAN ID**
  - Logical segmentation
  - Better security, Ex. Design policy such that only the nodes in a VLAN can communicate with each other

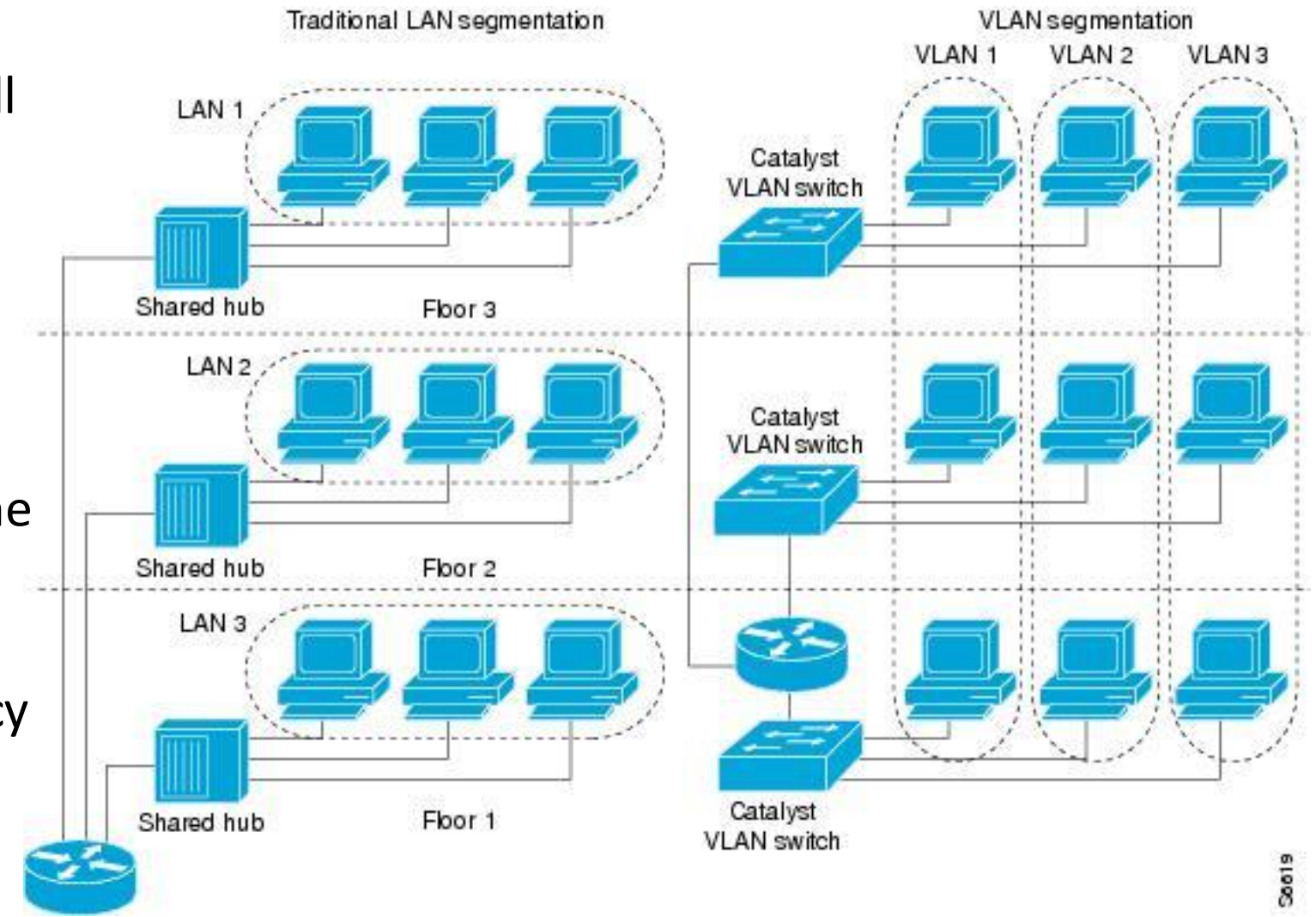
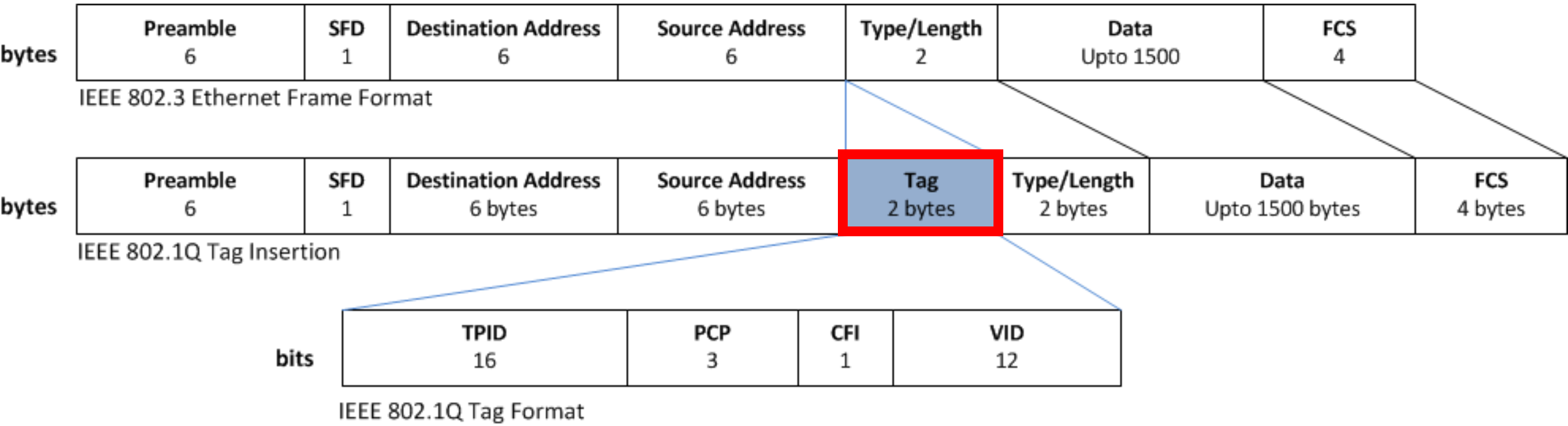


Image Source: https://www.cisco.com

VLAN Support in 802.1Q



TPID = Tag Protocol Identifier
PCP = Priority Code Point
CFI = Canonical Format Indicator
VID = VLAN Identifies (VLAN ID)